

Escrowbase - Document Management System

Overview

Escrowbase is a comprehensive escrow-based document management system designed to facilitate secure document handling, electronic signing, and collaboration in escrow transactions. Built on the ASP.NET Zero framework, it provides a robust, scalable solution for managing legal documents with multi-tenant architecture and enterprise-grade security.

The system integrates with DocuSign for electronic signatures, supports PDF viewing and annotation through PDFtron, and offers real-time collaboration features.

Features

Core Functionality

- **Document Management:** Upload, organize, and manage documents within escrow contexts
- **Electronic Signing:** Seamless integration with DocuSign for legally binding signatures
- **Collaboration Tools:** Invite users, assign roles, and manage permissions
- **Audit Trails:** Complete history of document access and modifications
- **Real-time Notifications:** SignalR-powered updates for collaborative workflows

Security & Compliance

- **Multi-tenant Architecture:** Isolated data per enterprise/client
- **Role-based Access Control:** Granular permissions for users and documents
- **Encryption:** Secure storage and transmission of sensitive documents
- **Compliance:** Designed to meet legal and regulatory requirements for escrow services

User Experience

- **Responsive Design:** Works on desktop, tablet, and mobile devices
- **PDF Viewer:** Advanced document viewing with annotation capabilities
- **Dashboard:** Centralized view of active escrows and tasks
- **Search & Filtering:** Powerful tools to find documents quickly

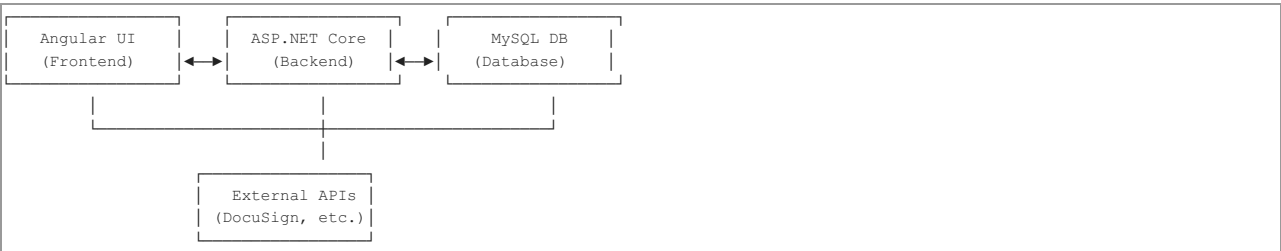
Technical Features

- **API-First Design:** RESTful APIs for integration with other systems
- **GraphQL Support:** Flexible data querying
- **Mobile Apps:** Native iOS and Android applications
- **Docker Deployment:** Containerized for easy deployment and scaling

Architecture

System Architecture Overview

The system follows a modern microservices-inspired architecture built on ASP.NET Zero:



Frontend Architecture

- **Framework:** Angular 12
- **UI Library:** PrimeNG, Bootstrap 4
- **Document Viewer:** PDFtron WebViewer
- **Charts:** ngx-charts, Chart.js
- **State Management:** Angular Services with RxJS
- **Build Tools:** Angular CLI, Gulp

Backend Architecture

- **Framework:** ASP.NET Core 3.1+
- **ORM:** Entity Framework Core
- **Authentication:** ASP.NET Identity with OAuth2/OIDC
- **Real-time:** SignalR
- **API:** RESTful with Swagger documentation
- **Background Jobs:** Hangfire (if configured)

Database Design

Core Entities

- **Enterprise:** Represents organizations using the system
- **EscrowClient:** Client entities within enterprises

- **SrEscrow**: Main escrow transaction entity
- **SrFileMapping**: Links files to escrows
- **SREscrowFileMaster**: Master file records
- **User**: System users with roles
- **E_SignRecord**: Electronic signature records
- **SrInvitationRecord**: Collaboration invitations

Relationships



Security Architecture

- **Authentication**: JWT tokens with refresh mechanism
- **Authorization**: ABP Framework's permission system
- **Data Protection**: Row-level security, encryption at rest
- **Audit Logging**: All operations tracked with user context

Installation & Setup

Prerequisites

- .NET 6.0 SDK
- Node.js 16+
- MySQL 8.0+
- Docker (optional)

Backend Setup

1. Clone the repository
2. Navigate to `aspnet-core` directory
3. Restore NuGet packages: `dotnet restore`
4. Update database connection in `appsettings.json`
5. Run migrations: `dotnet run --project SR.EscrowBaseWeb.Migrator`
6. Start the host: `dotnet run --project SR.EscrowBaseWeb.Web.Host`

Frontend Setup

1. Navigate to `angular` directory
2. Install dependencies: `npm install`
3. Configure API endpoint in `src/assets/appconfig.json`
4. Build and serve: `npm start`

Docker Deployment

```
# Build and run with Docker Compose
cd aspnet-core
docker-compose up -d
```

Usage Guide

Creating an Escrow

1. Log in as an enterprise user
2. Navigate to "New Escrow"
3. Enter escrow details and participants
4. Upload initial documents
5. Invite signatories via email

Document Management

1. Upload documents to escrow
2. Assign viewing/editing permissions
3. Use PDF viewer for annotations
4. Track document versions

Electronic Signing

1. Prepare document for signing
2. Send via DocuSign integration
3. Monitor signing status
4. Download completed documents

API Documentation

Authentication

```
POST /api/TokenAuth/Authenticate
Content-Type: application/json

{
  "userNameOrEmailAddress": "user@example.com",
  "password": "password"
}
```

Escrow Operations

- GET /api/services/app/SrEscrows/GetAll - List escrows
- POST /api/services/app/SrEscrows/Create - Create escrow
- PUT /api/services/app/SrEscrows/Update - Update escrow

File Operations

- POST /api/services/app/SrFileMappings/UploadFile - Upload file
- GET /api/services/app/SrFileMappings/GetFile/{id} - Download file

System Design Details

Workflow Engine

The system implements a state-based workflow for escrow processing:

1. **Draft:** Initial creation
2. **Active:** Participants invited
3. **Signing:** Documents sent for signature
4. **Completed:** All signatures collected
5. **Closed:** Escrow finalized

Notification System

- Email notifications for invitations and updates
- In-app notifications via SignalR
- SMS integration (configurable)

File Storage

- Secure cloud storage integration
- Local file system with encryption
- CDN for fast access

Performance Considerations

- Database indexing on frequently queried fields
- Caching layer for metadata
- Asynchronous processing for heavy operations
- Horizontal scaling support

Development

Project Structure

```
Escrowbase/
├── angular/           # Frontend application
│   ├── src/
│   │   ├── app/
│   │   └── assets/
│   └── package.json
├── aspnet-core/      # Backend application
│   ├── src/
│   │   ├── SR.EscrowBaseWeb.Application/
│   │   ├── SR.EscrowBaseWeb.Core/
│   │   ├── SR.EscrowBaseWeb.EntityFrameworkCore/
│   │   └── SR.EscrowBaseWeb.Web.Host/
│   └── test/
└── docker/           # Docker configurations
```

Testing

- Unit tests with xUnit (backend)
- Integration tests with TestServer
- E2E tests with Protractor (frontend)
- Manual testing guidelines

Deployment Pipeline

1. Code commit triggers CI/CD
2. Automated testing
3. Build artifacts
4. Deploy to staging

5. Manual approval for production

Contributing

1. Fork the repository
2. Create a feature branch
3. Make changes with tests
4. Submit pull request
5. Code review and merge

Code Standards

- Follow ASP.NET Zero coding guidelines
- Use ESLint/TSLint for frontend
- Maintain test coverage >80%
- Document public APIs

License

This project is licensed under the MIT License - see the LICENSE file for details.

Support

For support and questions:

- Documentation: [Link to docs]
- Issues: GitHub Issues
- Email: support@escrowbase.com

Generated on April 15, 2026</content>

<parameter name="filePath">c:\Escrowbase_2025\Escrowbase_Documentation.md